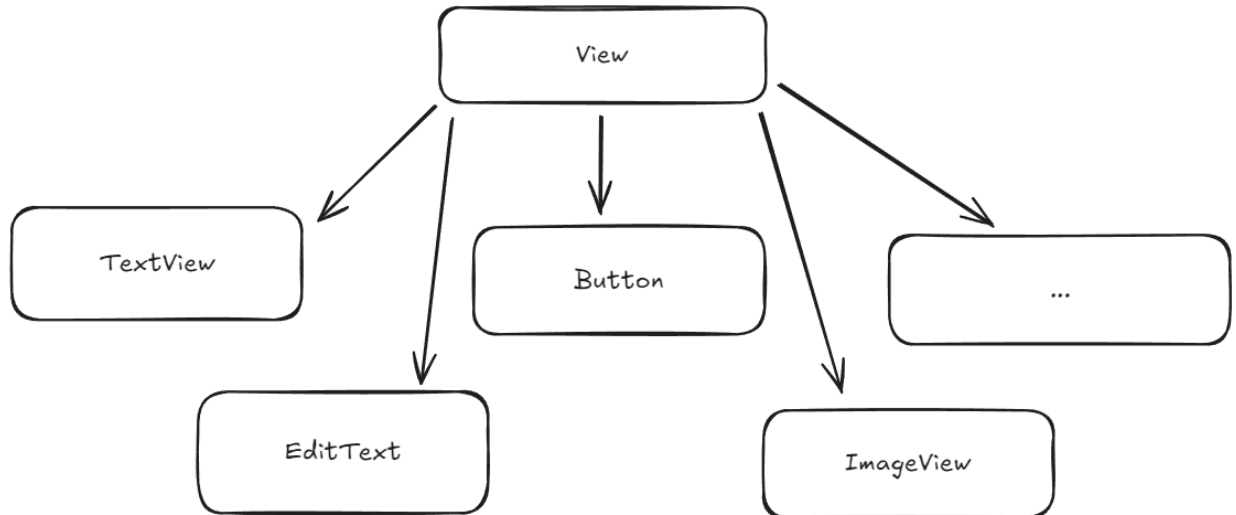


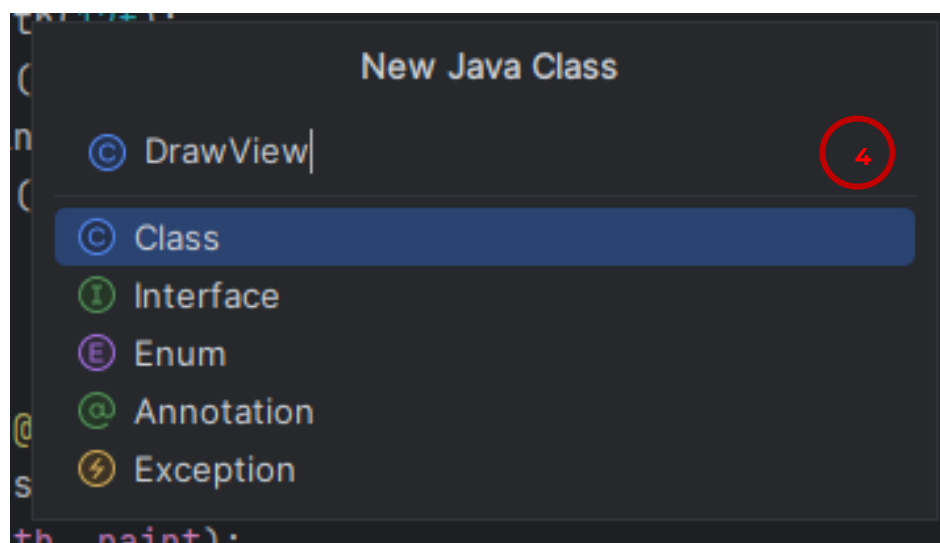
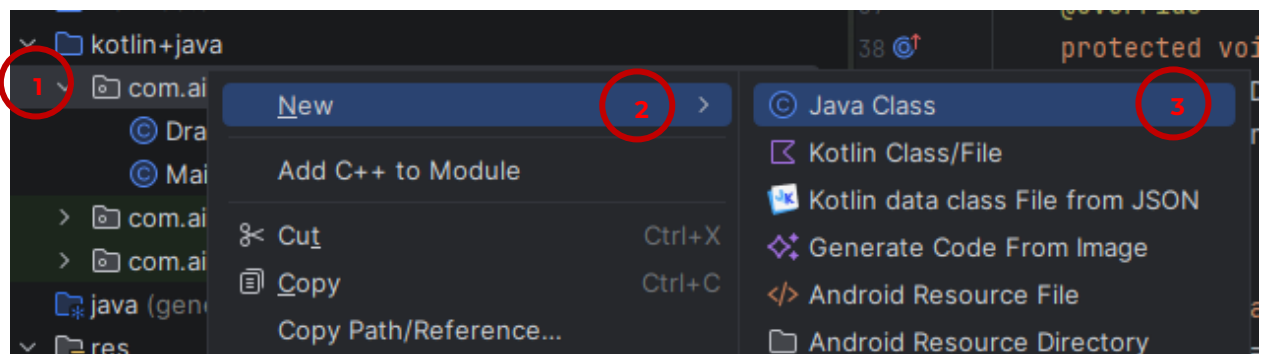
ЛАБОРАТОРНАЯ РАБОТА №5 РАЗРАБОТКА ПРИЛОЖЕНИЯ ДЛЯ РИСОВАНИЯ НА ЭКРАНЕ

1. Собственный элемент интерфейса

Все элементы интерфейса, которые вы использовали раньше (**Button**, **TextView**, **ImageView**) – это классы, наследующие View.



Никто не мешает создать **свой класс-наследник** View и научить его делать то, что нужно именно нам, например, **рисовать**. Такой компонент можно многократно использовать в разных разметках и приложениях.



В пакете проекта создайте класс **DrawView** и унаследуйте его от **View**:

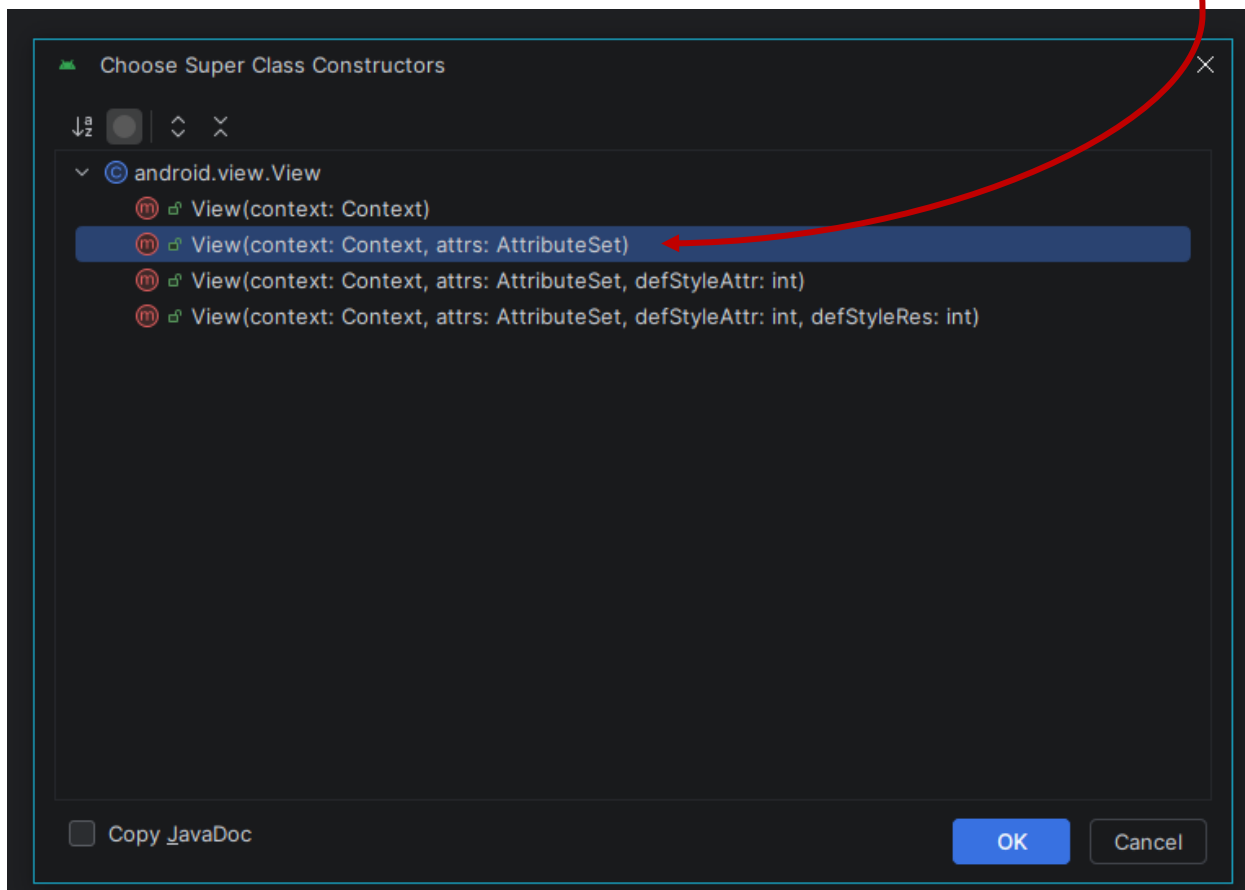
```
5 public class DrawView extends View {  
6  
7 }  
8 |
```

Так как мы **унаследовались** от класса View – Android Studio просит нас создать **конструктор**:

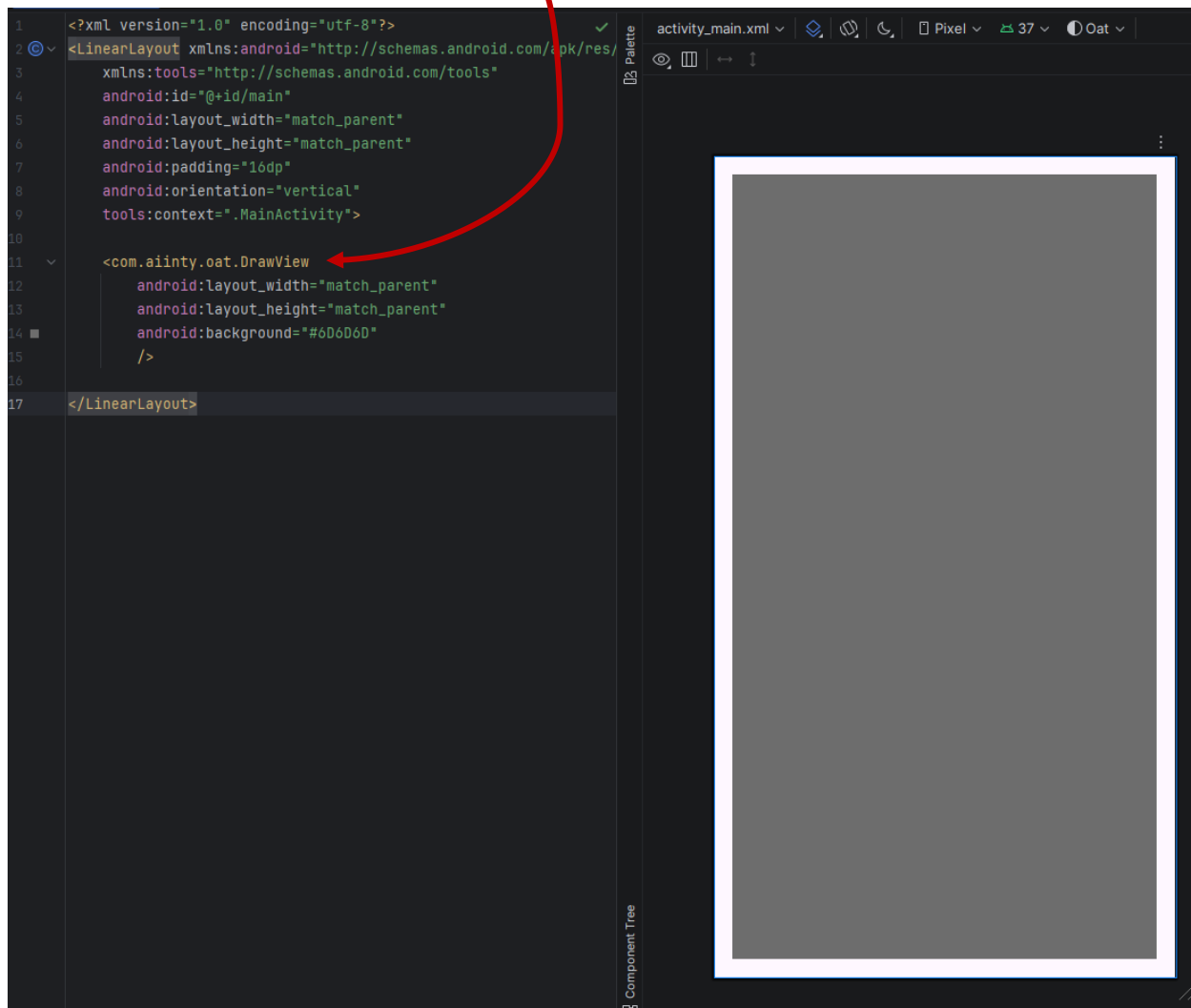
```
public class DrawView extends View {  
}  
}
```

Create constructor matching super
Create subclass
Create test

Обязательно выбираем **второй**, так как именно он вызывается при создании нашего элемента в XML.



Теперь попробуйте поместить элемент в **разметку экрана**:



Пока что на экране ничего нет (кроме **background**, который мы задали, чтоб было видно элемент).

2. Первая отрисовка: Canvas и Paint

За рисование отвечают два класса:

1. **Canvas** (холст) – поверхность, на которой рисуют (методы `drawCircle`, `drawLine`, `drawPath`)
2. **Paint** (кисть) – цвет, стиль, толщина линий

Отрисовка элемента выполняется в методе **`onDraw(Canvas canvas)`**. Система вызывает его **каждый раз**, когда элемент нужно нарисовать или перерисовать.

Рисовать можно только внутри `onDraw` – запомните это правило, оно пригодится в части 3.

Дополните класс **DrawView**:

```
12 public class DrawView extends View {
13
14     private Paint paint; 9 usages
15
16     public DrawView(Context context, @Nullable AttributeSet attrs) {
17         super(context, attrs);
18         setupPaint();
19     }
20
21     private void setupPaint() { 1 usage
22         paint = new Paint();
23         paint.setColor(Color.BLUE); // цвет кисти
24         paint.setStyle(Paint.Style.FILL); // FILL - заливка фигуры
25         paint.setAntiAlias(true); // сглаживание краёв
26     }
27
28     @Override
29     protected void onDraw(Canvas canvas) {
30         super.onDraw(canvas);
31         // drawCircle(x, y, радиус, кисть)
32         canvas.drawCircle(cx: getWidth() / 2f, cy: getHeight() / 3f, radius: 100, paint);
33
34         paint.setColor(Color.RED);
35         canvas.drawCircle(cx: getWidth() / 2f, cy: getHeight() / 2f, radius: 150, paint);
36
37         paint.setColor(Color.rgb(red: 76, green: 175, blue: 80));
38         canvas.drawCircle(cx: getWidth() / 2f, cy: getHeight() * 2 / 3f, radius: 200, paint);
39     }
40 }
41
```

Обратите внимание: координаты берутся через **`getWidth()`** и **`getHeight()`** – размеры самого элемента. Так круги окажутся по центру на любом экране, а не где-то слева сверху на устройствах с другой плотностью пикселей.

Запустите приложение – должны появиться **три круга разных цветов**. Поэкспериментируйте с координатами, радиусами и методом **`setColor`** (можно задавать цвет и по имени: `Color.RED`, `Color.GREEN`).

Попробуйте заменить **`Paint.Style.FILL`** на **`Paint.Style.STROKE`** – вместо залитых кругов получите окружности.

3. Реакция на касание: точки

А если нам нужно, чтобы рисовал пользователь? Когда палец касается элемента, система вызывает метод **onTouchEvent()**, передавая объект **MotionEvent** – из него можно получить координаты касания.

Задача: при **каждом касании добавлять** точку на холст. Мы заранее не знаем ни количество точек, ни их координаты, поэтому хранить их будем в списке.

Создайте новый класс **DrawPointsView** и измените его:

```
17 public class DrawPointsView extends View {
18
19     // Список точек, добавляемых касанием. final - ссылка не меняется,
20     // но элементы в список добавлять можно
21     private final List<Point> points = new ArrayList<>>(); 2 usages
22     private Paint paint; 5 usages
23
24     public DrawPointsView(Context context, @Nullable AttributeSet attrs) {
25         super(context, attrs);
26         setupPaint();
27     }
28
29     private void setupPaint() { 1 usage
30         paint = new Paint();
31         paint.setColor(Color.rgb(red: 103, green: 58, blue: 183));
32         paint.setStyle(Paint.Style.FILL);
33         paint.setAntiAlias(true);
34     }
35
36     @Override
37     protected void onDraw(Canvas canvas) {
38         super.onDraw(canvas);
39         // рисуем точку для каждого элемента списка:
40         for (Point p : points) {
41             canvas.drawCircle(p.x, p.y, radius: 10, paint);
42         }
43     }
44
45     @Override
46     public boolean onTouchEvent(MotionEvent event) {
47         // Координаты касания относительно нашего элемента
48         points.add(new Point((int) event.getX(), (int) event.getY()));
49         // Сообщаем системе, что элемент нужно перерисовать
50         postInvalidate();
51         return true; // = событие обработано
52     }
53 }
54
```

postInvalidate() запускает перерисовку. Сам по себе **onTouchEvent** не рисует — он только меняет данные (у нас это список точек). Рисует всегда **onDraw**.

Метод **postInvalidate()** говорит системе: «данные изменились, вызови **onDraw** ещё раз».

4. Плавные линии: Path

Точки рисуются с промежутками: каждая это **отдельная несвязанная** фигура. Рисовать пальцем отдельными точками неудобно.

Для непрерывных линий подходит класс **Path** – контур, который может содержать множество линий и кривых.

Создайте класс **DrawPathView**:

```
public class DrawPathView extends View {
    private Paint paint; 8 usages
    private Path path; 4 usages

    public DrawPathView(Context context, @Nullable AttributeSet attrs) {
        super(context, attrs);
        setupPaint();
    }

    private void setupPaint() { 1 usage
        path = new Path();
        paint = new Paint();
        paint.setColor(Color.rgb( red: 233, green: 30, blue: 99));
        paint.setStyle(Paint.Style.STROKE); // рисуем контур, не заливку
        paint.setStrokeWidth(12f); // толщина линии
        paint.setStrokeCap(Paint.Cap.ROUND); // скруглённые концы линий
        paint.setStrokeJoin(Paint.Join.ROUND); // скруглённые соединения
        paint.setAntiAlias(true);
    }

    @Override
    protected void onDraw(@NonNull Canvas canvas) {
        super.onDraw(canvas);
        canvas.drawPath(path, paint);
    }

    @Override
    public boolean onTouchEvent(MotionEvent event) {
        float x = event.getX();
        float y = event.getY();

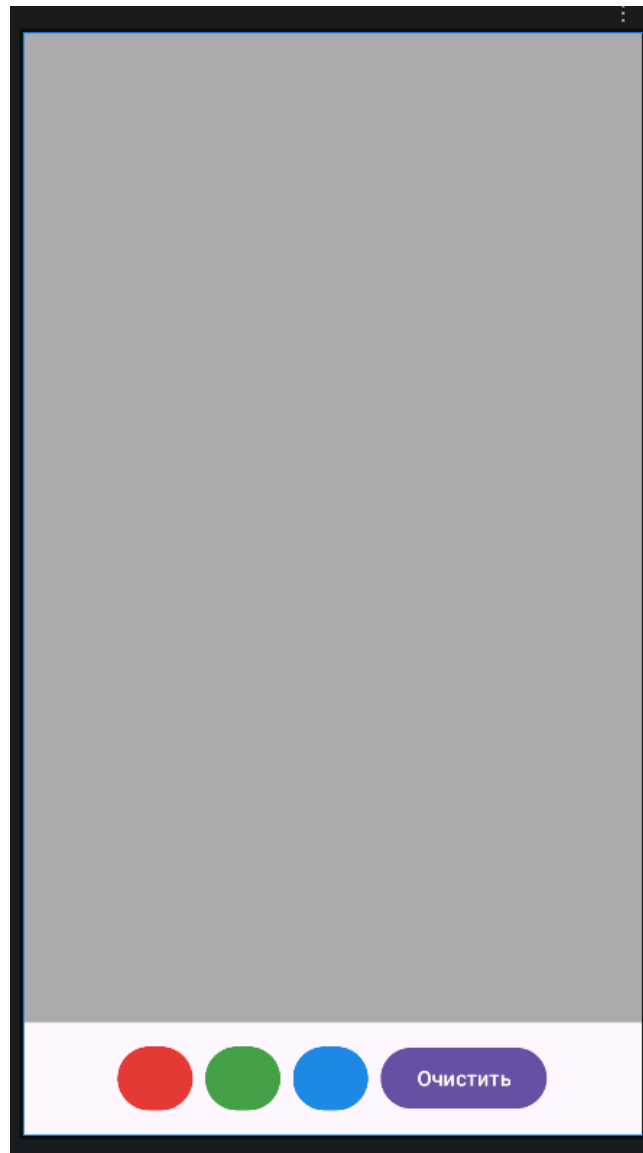
        switch (event.getAction()) {
            case MotionEvent.ACTION_DOWN:
                path.moveTo(x, y); // начать новый штрих в точке касания
                break;
            case MotionEvent.ACTION_MOVE:
                path.lineTo(x, y); // продолжить линию до новой точки
                break;
            case MotionEvent.ACTION_UP:
                break; // штрих закончен – ничего делать не нужно
            default:
                return false;
        }
        postInvalidate();
        return true;
    }
}
```

Проверьте: линия следует за пальцем плавно, без разрывов.

Заметьте: **moveTo** на ACTION_DOWN означает, что новый штрих не соединяется с предыдущим – можно рисовать несколько отдельных линий.

5. Дополнительно

Сейчас компонент всего один, и цвет кисти защиты в код. Доведем приложение до конца: добавьте панель переключения цвета и очистку:



Чтобы компонентом можно было управлять извне, добавьте в **DrawPathView** публичные методы – это “пульт управления” вашим компонентом:

```
public void setColor(int color) {  
    paint.setColor(color);  
}  
  
public void clear() { no usages  
    path.reset();      // очистить контур  
    postInvalidate();  // и перерисоваться  
}
```